Product Discovery and Prioritisation: A Practical Guide

What product discovery is

Product discovery is the work of deciding what to build before you build it. Its goal is to reduce the risk of shipping something nobody wants. A good discovery process answers four questions: is the problem real, is it valuable to solve, can we build the solution, and will it work for the business. Teams that skip discovery tend to build features that look impressive but move no real metric.

Talking to users

The single most reliable input to discovery is regular conversations with real users. Aim for a small number of interviews every week rather than one big research project once a year. Ask about what people actually did, not what they say they would do. Watch for the workarounds and spreadsheets people build themselves, because those are signals of unmet needs you can turn into product opportunities.

Defining the real problem

The Jobs To Be Done idea says people hire a product to make progress in a situation. Describe the job in the user's terms, not the feature's terms. For example, the job is not 'use our dashboard'; the job is 'know on Monday morning whether last week went well'. Framing the problem this way keeps the team focused on the outcome and open to more than one possible solution.

Prioritisation: value versus effort

The simplest prioritisation tool is a two-by-two of value against effort. High-value, low-effort work is done first; low-value, high-effort work is dropped. It is fast and good for a rough first pass, but it hides assumptions, so use it to start a discussion rather than to end one.

The RICE framework

RICE scores each idea on Reach, Impact, Confidence, and Effort. Reach is how many users are affected in a period. Impact is how much it moves the goal per user. Confidence is how sure you are, expressed as a percentage. Effort is the person-time required. The score is Reach times Impact times Confidence, divided by Effort. RICE is useful because it forces you to state your confidence out loud instead of hiding it.

The Kano model

The Kano model sorts features into three kinds. Basic expectations cause anger when missing but no delight when present, such as a login that works. Performance features are liked more the more you have of them, such as speed. Delighters are unexpected pleasers that create loyalty. A healthy release covers the basics first, then invests in a few delighters, rather than adding performance features nobody asked for.

MoSCoW

MoSCoW splits scope into Must have, Should have, Could have, and Won't have this time. It is most useful for scoping a specific release under a deadline, because naming the explicit 'won't have' list is what actually protects the timeline. Without a 'won't have' list, scope quietly grows until the date slips.

Building a roadmap

A roadmap is a statement of intent, not a delivery contract. Prefer a roadmap organised around problems and outcomes for the next quarter rather than a dated list of features for the next year. Group work into now, next, and later. This keeps you honest that far-away items are guesses and gives you room to change course as you learn.

Common prioritisation mistakes

The most common mistake is prioritising the loudest request instead of the most valuable one. The second is treating every framework score as a fact rather than an estimate. The third is never revisiting the list, so stale priorities drive the roadmap for months. Re-score at a regular cadence and treat the numbers as a conversation starter.